

RTS Demo - Persistence Layer Technical Specification

Overview

The persistence layer provides game state serialization and deserialization via LiteDB, an embedded NoSQL database. The system captures unit positions, health, animation states, and combat status to `Application.persistentDataPath/RTS_demo_saves.db` and restores complete game snapshots on demand.

Property	Value
Database Engine	LiteDB (embedded, zero-configuration)
Storage Path	<code>Application.persistentDataPath/RTS_demo_saves.db</code>
Collection	"saves" (indexed on timestamp)
Access Pattern	Synchronous upsert/retrieve by slot identifier
Current Slots	"quicksave" (extensible to multiple slots)

Architecture

GameManager.QuickSave/Load (F5/F9)
 ↳
DatabaseManager (LiteDB wrapper)
 ↳
MapStateSaveData + UnitSaveData[]
 ↳
LiteDB Collection "saves"

Save Flow: Iterate allUnits ↳ extract to UnitSaveData[] ↳ pack into MapStateSaveData ↳ upsert to database

Load Flow: Query slot ↳ iterate UnitSaveData[] ↳ match by unitId ↳ restore transform/health/state
 ↳ 0.5s stabilization delay

Data Structures

MapStateSaveData

Root container for complete game state snapshot.

Field	Type	Purpose
saveTime	DateTime	Timestamp of snapshot creation
allUnits	UnitSaveData[]	Array of all unit states

UnitSaveData

Per-unit serializable state.

Field	Type	Purpose
id	int	Index in allUnits[]
unitId	string	Unique unit identifier (game-assigned)
unitType	string	"SwatGuy" or "HeavyTank" (enum-as-string)
team	string	"Red", "Blue", or "Neutral"
positionX/Y/Z	float	World position (decomposed from Vector3)
rotationX/Y/Z	float	Euler angles (decomposed from Quaternion)
destinationX/Y/Z	float	NavMeshAgent target destination
currentHealth	int	Current HP
maxHealth	int	Maximum HP baseline
isAlive	bool	Alive flag
isMoving	bool	Locomotion state
isAttacking	bool	Combat engagement state
attackCooldownRemaining	float	Seconds until next attack available
targetId	string	Currently targeted unit ID (null if none)

Public API

DatabaseManager

Method	Signature	Purpose
QuickSave()	void QuickSave(MapStateSaveData state, string slot = "quicksave")	Upsert state to database slot (atomic)
QuickLoad()	MapStateSaveData QuickLoad(string slot = "quicksave")	Retrieve state from slot (returns null if not found)
DeleteSave()	void DeleteSave(string slot = "quicksave")	Remove save slot from database

CreateMapState()	MapStateSaveData CreateMapState()	Factory: initialize empty container with current timestamp
------------------	-----------------------------------	--

Lifecycle: Awake() initializes LiteDB connection; OnDestroy() closes and disposes resources.

GameManager

Method	Trigger	Purpose
QuickSave()	F5 key	Capture current game state and persist to database
QuickLoad()	F9 key	Retrieve last save and restore all unit states
CaptureMapState()	Invoked by QuickSave()	Iterate allUnits[], extract to UnitSaveData[]
LoadMapState()	Invoked by QuickLoad()	Clear dead units, restore transforms/health/targets, trigger physics stabilization

Operational Details

Save Operation

1. Player presses F5
2. CaptureMapState() iterates allUnits, extracting each unit's transform, health, and combat state to UnitSaveData
3. DatabaseManager.QuickSave() performs atomic upsert to slot "quicksave"
4. LiteDB writes to disk (brief synchronous blocking, ~15-60ms)

Load Operation

1. Player presses F9
2. DatabaseManager.QuickLoad() retrieves MapStateSaveData from "quicksave" slot
3. LoadMapState()
executes:
 - Clear dead units from allUnits list
 - Iterate loaded UnitSaveData[], match by unitId to current units
 - For each matched unit: disable NavMeshAgent → set transform position/rotation → call Warp() for physics sync → re-enable NavMeshAgent → restore health/state flags
 - Trigger Unit.StabilizeAfterLoad() coroutine (0.5s delay for collision settling)

Performance Profile

Operation	Duration	Notes
CaptureMapState (50 units)	5-10ms	Linear scan
Database upsert	10-50ms	Disk I/O; synchronous
Full save	15-60ms	Possible minor frame stutter
Database retrieval	1-2ms	Indexed lookup
LoadMapState restoration	5-10ms	Unit iteration and state restoration
Physics stabilization	500ms	Intentional delay; prevents collision glitches

Expansion Points

Multiple Save Slots

Current State: Single "quicksave" slot **Extension:** Pass optional slotName parameter to QuickSave(state, "autosave") and QuickLoad("slot_1"); add UI dropdown for slot selection

Building Persistence

Current State: Only units serialized **Extension:** Add BuildingData[] allBuildings to MapStateSaveData with identical field pattern (id, position, rotation, health, team)

Resource/Economy State

Current State: No resource counts saved **Extension:** Add ResourceState resourceState to MapStateSaveData containing per-team resource counts

Event Replay

Current State: Snapshot-only saves (no input history) **Extension:** Record GameInputEvent[] (timestamp, inputType, targetPosition, unitId) for deterministic replay

Implementation Notes

- **Vector3 Decomposition:** Position and rotation stored as individual floats (positionX, Y, Z) rather than nested objects for LiteDB compatibility and clarity
- **Enum Serialization:** Enums converted to strings during capture ("SwatGuy", "Red") for human-

readable storage

- **Target Resolution:** Target units saved by string ID; resolution back to Unit references deferred to load-time lookup
- **Null Safety:** Dead/missing units skipped during capture; unmatched units ignored during load
- **Physics Synchronization:** NavMeshAgent disabled during teleport, Warp() called for physics sync, then re-enabled; 0.5s coroutine delay prevents collision detection glitches

Storage Location

Platform	Path
Windows	C:\Users\[User]\AppData\LocalLow\[Company]\[Product]\RTS_demo_saves.db
macOS	~/Library/Application Support/[Company]/[Product]/RTS_demo_saves.db
Linux	~/.config/unity3d/[Company]/[Product]/RTS_demo_saves.db

Resolved via Application.persistentDataPath at initialization.

Performance Characteristics

Impact on Framerate

Operation	Duration	Impact
Save cycle (F5, 50 units)	15-60ms	Minor frame stutter possible at 60fps
Capture state (50 units)	5-10ms	Linear scan complexity
Database upsert	10-50ms	Disk I/O varies by system speed
Load cycle (F9, 50 units)	506-512ms	500ms is intentional physics delay
Database retrieval	1-2ms	Indexed slot lookup
Unit restoration	5-10ms	Per-unit state assignment

Database Growth

Scenario	Per-Unit	Per-Slot	Notes
Minimal (10 units)	150-250 bytes	2-5 KB	Compressed LiteDB storage
Medium (50 units)	200-350 bytes	10-18 KB	Typical sandbox configuration
Maximum (100+ units)	250-400 bytes	25-40 KB	Performance may degrade

Multiple slots accumulate linearly; no deduplication across slots.

Limitations & Constraints

Limitation	Current State	Workaround
Single save mechanism	Only QuickSave/QuickLoad available	Use slot parameter for multiple saves
Synchronous I/O	Blocking disk operations	Defer saves to non-critical frames
No target re-linking	Targets saved as ID strings, not restored to Unit references	Manual re-targeting post-load or future lookup implementation
No building/terrain state	Only units persisted	Extend MapStateSaveData with additional arrays
No input history	Snapshots only, no replay capability	Implement GameInputEvent recording separately
Data not encrypted	Plain binary LiteDB format	For production, wrap with AES encryption layer
Linear unit matching	O(n) search to match units by ID	For 1000+ units, consider indexed unit registry

Error Handling

API Behaviors

Method	Failure Scenario	Return Value	Log Level
QuickLoad()	Slot not found	null	Warning
QuickLoad()	Database corrupted	Exception (propagated)	Error (console)
QuickSave()	Disk full	Exception (LiteDB propagated)	Error (console)
CaptureMapState()	Unit list modified during iteration	Skips null entries	Warning (per unit)
LoadMapState()	Unit ID not found in current scene	Skips unmatched unit	Warning (per unit)

Recommendation: Wrap save/load calls in try-catch for production UI.

Recovery Procedures

Issue	Recovery
Database file corrupted	Delete .db file; application creates fresh database on next save
Load fails with null	Verify save slot was created (press F5 first)
Units at origin (0,0,0) after load	Old save with invalid positions; create new save
NavMeshAgent blocked post-load	Physics not stabilized; wait 0.5s before issuing commands

Debugging

All save/load operations emit Debug.Log messages with prefixes for Console filtering:

Prefix	Component	Example
[DB]	DatabaseManager	[DB] Quicksave quicksave at 1/21/2026 14:32:45
[SAVE]	CaptureMapState	[SAVE] Unit unit_Red_1 at (10.50, 1.20, 8.30)
[LOAD]	LoadMapState	[LOAD] Loading 2 units...

Usage: Filter Console window by prefix to isolate persistence layer traffic.

File References

File	Purpose
DatabaseManager.cs	LiteDB wrapper; manages connection and collection access
SaveData.cs	UnitSaveData and MapStateSaveData class definitions
GameManager.cs	QuickSave/Load orchestration; CaptureMapState and LoadMapState implementation
Unit.cs	StabilizeAfterLoad() coroutine